Table 1: Case studies given in the library

Case study and script	Key functions and parameters	Zero Trust security outcome
Context drift test context-drift-test.cc	*StoreBaseline()* *UpdateCurrentContext()* *SetParameters()* *Revalidate()* *ZtPolicyEngine::Revoke()*	Continuously calculates weighted drift and decays trust *(Trust = Trust × e^(-λ × drift))*. Automatically revokes Node 3 and Node 4 when trust falls below 0.40.
Micro-segmentation test micro-segmentation-test.cc	*RegisterNode()* *AddRolePolicyRule()* *EvaluateMicroSegmentation()* *ComputeSha256()*	Generates policy fingerprint Ψ. Grants normal streams, blocks excess streaming (>5 packets), denies maintenance updates outside allowed hours, and defaults unapproved actions to DENY under least privilege.
Dynamic trust scorer test trust-scorer-test.cc	*OnPacketReceived()* *IsSessionAllowed()* *SendNormal()* *SendBurst()*	Penalises trust exponentially during rate violations. Allows gradual score recovery during compliant traffic. Hard-blocks the session once trust drops below cutoff score 25, dropping all incoming packets.

contextual metrics: time of day, network congestion, and proximity, captured at onboarding. The *PeriodicRevalidator* regularly computes weighted drift and reduces node trust using exponential decay: *Trust = Trust × e^(-λ × drift)*, calling *ZtPolicyEngine::Revoke* whenever trust drops below a designated threshold.

Dynamic trust scorer *(ZtDynamicTrustScorer):* Provides session-level anomaly detection between communication pairs. By tracking packet arrival rates across a sliding window through *OnPacketReceived*, it flags violations exceeding 1.5 times the baseline, penalises consecutive bursts exponentially, permits gradual recovery for compliant traffic, and blocks sessions once trust falls below 25.

Encryption utilities *(EncryptPayload, DecryptPayload, HexDecodeKey, HexToBytes)*: Wraps Crypto++ AES-CBC mode to encrypt data payloads with a randomly generated 16-byte initialisation vector prepended to the ciphertext, ensuring transport confidentiality.

TLS handshake *(ZtTlsHandshake):* Coordinates a policy-gated mutual handshake between client and server nodes. Before generating symmetric session keys, it invokes an injected policy validation callback backed by *ZtPolicyEngine::Authorize*; once cleared, it generates and stores a shared AES key accessible via *GetSessionKey*.

IPv4 enforcement hook *(ZtIpv4Hook):* Connects directly to an ns3::Node IPv4 stack to intercept outgoing transmission traces (*Tx*), translating abstract micro-segmentation decisions into real-time network-layer packet forwarding or dropping like an active policy enforcement point (PEP).

Logger *(ZtLogger):* Offers a unified static logging interface featuring optional timestamping, standardising console diagnostics across certificate issuance, validation failures, revocations, and payload cryptographic operations.

Table 1 outlines the three standalone case studies provided in the library, summarising their test topology, functions used, configured rules, and security enforcement outcomes.

Table 2 contrasts three attack simulations without Zero Trust against their mitigated counterparts using the developed library.

Implementing Zero Trust in IoT networks replaces outdated implicit trust models with strict, continuous verification and least privilege access. Through ns-3 simulations, developers and researchers can easily evaluate how cryptographic handshakes, fine-grained micro-segmentation policies and runtime context monitoring safeguard vulnerable smart devices.

The ten integrated modules of the Zero Trust IoT library successfully

Release history and open source contributions

The zero-trust-iot library has evolved across three key public releases on the NS-3 App Store. Version 1.0.0 established the foundational Zero Trust framework compatible with ns-3.45 using Crypto++ primitives. The latest Version 2.0.0 for ns-3.47 expands the suite with five advanced modules covering certificate authorities, dynamic trust scoring, and context drift validation. Distributed under the open source GPLv2 license, the maintainers actively invite community contributions, feedback, and new IoT attack-defence simulation models through its official GitHub repository and issue tracker.